1. Which keyword in Python is used for conditional multi-way branching (else-if)?

(A) elif
(B) elseif
(C) else if
(D) elsif

2. In Python, which character must terminate the header line of an if, elif, else, for, while, or def statement?

(A) : (colon)
(B) ; (semicolon)
(C) { (left curly brace)
(D) -> (arrow)

3. Which statement in Python represents the ternary conditional operator syntax?

(A) value_if_true if condition else value_if_false
(B) condition ? value_if_true : value_if_false
(C) if(condition, true, false)
(D) value_if_true when condition else value_if_false

4. Which loop in Python is typically used to iterate over a sequence (such as a range, list, tuple, or string)?

(A) for loop
(B) while loop
(C) do-while loop
(D) repeat-until loop

5. Which built-in function generates an immutable sequence of integers commonly used for loop counting?

(A) range()
(B) sequence()
(C) count()
(D) series()

6. What sequence of integers is generated by range(5)?

(A) 0, 1, 2, 3, 4
(B) 1, 2, 3, 4, 5
(C) 0, 1, 2, 3, 4, 5
(D) 1, 2, 3, 4

7. What sequence of integers is generated by range(2, 10, 2)?

(A) 2, 4, 6, 8
(B) 2, 4, 6, 8, 10
(C) 4, 6, 8, 10
(D) 2, 3, 4, 5, 6, 7, 8, 9

8. Which loop in Python executes its block of statements repeatedly as long as a test condition remains True?

(A) while loop
(B) for loop
(C) loop loop
(D) switch loop

9. Which jump statement immediately terminates the execution of the innermost loop?

(A) break
(B) continue
(C) pass
(D) exit

10. Which jump statement skips the rest of the current iteration and jumps directly to the next iteration of the loop?

(A) continue
(B) break
(C) skip
(D) next

11. Which keyword serves as a syntactical placeholder statement where code is required but no action should be performed?

(A) pass
(B) null
(C) noop
(D) continue

12. In Python, an optional 'else' clause attached to a for or while loop executes:

(A) When the loop completes all iterations normally without encountering a break statement
(B) When a break statement is triggered
(C) On every iteration
(D) Only if an error occurs

13. Which keyword is used to define a reusable function in Python?

(A) def
(B) function
(C) func
(D) sub

14. What is the difference between a Function Parameter and a Function Argument in Python?

(A) Parameters are variable names in the function definition; Arguments are concrete values passed during invocation
(B) Arguments are in definition; parameters in call
(C) They are 100% synonymous in all contexts
(D) Parameters are always global

15. What value does a Python function return by default if no explicit return statement is executed?

(A) None
(B) 0
(C) False
(D) Empty string ''

16. How do you define a default parameter value in a function header?

(A) def func(param=default_val):
(B) def func(param : default_val):
(C) def func(default param = val):
(D) def func(param := val):

17. Why must parameters with default values always follow non-default parameters in a function definition?

(A) Positional arguments cannot follow default arguments due to ambiguity in argument binding
(B) Python compiler forbids defaults
(C) It causes stack overflows
(D) It requires 64-bit architecture

18. Which syntax allows a Python function to accept an arbitrary variable number of positional arguments?

(A) *args (prefixed with a single asterisk)
(B) **kwargs
(C) *params
(D) ...args

19. Inside a function, what data structure type does the *args parameter arrive as?

(A) tuple
(B) list
(C) dict
(D) set

20. Which syntax allows a function to accept an arbitrary variable number of keyword arguments?

(A) **kwargs (prefixed with double asterisks)
(B) *args
(C) **dict
(D) kwargs()

21. Inside a function, what data structure type does the **kwargs parameter arrive as?

(A) dict (Dictionary)
(B) tuple
(C) list
(D) set

22. What is a Lambda Function in Python?

(A) An anonymous, small inline function defined using the 'lambda' keyword restricted to a single expression
(B) A function executed in a separate thread
(C) A function that returns complex numbers
(D) A recursive constructor

23. What is the syntax for defining an anonymous square function using lambda?

(A) square = lambda x: x ** 2
(B) square = def(x) -> x ** 2
(C) square = fn(x): x ** 2
(D) square = (x) => x ** 2

24. What is Recursion in computer programming?

(A) A programming technique where a function calls itself directly or indirectly to solve smaller instances of a problem
(B) Iterating with a while loop
(C) Importing external modules
(D) Multithreaded task scheduling

25. What critical condition must every recursive function possess to prevent infinite recursion?

(A) A Base Case that stops further recursive calls
(B) A while loop
(C) A global variable
(D) A lambda expression

26. What built-in exception is raised when Python exceeds its maximum call stack recursion depth?

(A) RecursionError (Maximum recursion depth exceeded)
(B) StackOverflowError
(C) MemoryError
(D) DepthError

27. Which module and function in Python allows inspecting and setting the recursion depth limit?

(A) sys.getrecursionlimit() and sys.setrecursionlimit()
(B) os.getrecursion()
(C) math.recursivelimit()
(D) gc.limit()

28. What is the default recursion depth limit in standard CPython 3?

(A) 1000 stack frames
(B) 100 stack frames
(C) 10,000 stack frames
(D) Infinite

29. Which keyword inside a function informs Python that a variable assignment refers to a module-level global variable?

(A) global
(B) nonlocal
(C) extern
(D) public

30. Which keyword inside a nested function refers to a variable in the nearest enclosing non-global scope?

(A) nonlocal
(B) global
(C) outer
(D) super

31. What structural pattern matching statement was introduced in Python 3.10 to replace multi-level if-elif chains?

(A) match ... case
(B) switch ... case
(C) select ... case
(D) choose ... when

32. In a Python 3.10 match-case statement, which wildcard symbol represents the default fallback case?

(A) _ (underscore)
(B) default
(C) *
(D) else

33. Which built-in higher-order function applies a function to all items in an iterable, returning an iterator?

(A) map(function, iterable)
(B) filter()
(C) reduce()
(D) apply()

34. Which built-in function extracts elements from an iterable for which a test function returns True?

(A) filter(function, iterable)
(B) map()
(C) select()
(D) extract()

35. Which module must be imported in Python to use the reduce() functional accumulator function?

(A) functools (from functools import reduce)
(B) itertools
(C) math
(D) collections

36. What is a Docstring in a Python function, and where is it placed?

(A) A triple-quoted documentation string placed immediately on the first line inside the function body
(B) A comment written before def
(C) A string in requirements.txt
(D) A variable type annotation

37. Which special attribute on a function object exposes its docstring documentation?

(A) func.__doc__
(B) func.doc
(C) func.__help__
(D) func.description

38. What are Function Annotations / Type Hints introduced in PEP 3107 and PEP 484 (e.g., `def calc(x: float) -> float:`)?

(A) Syntactical hints indicating expected argument and return types, accessible via `__annotations__` without runtime enforcement
(B) Strict runtime type compilation rules
(C) C++ compiler pragmas
(D) Memory layout directives

39. Which built-in function pairs each item in an iterable with an automatic incrementing index counter (0, 1, 2...)?

(A) enumerate(iterable, start=0)
(B) zip()
(C) counter()
(D) index()

40. Which built-in function aggregates elements from two or more iterables into tuples pairwise?

(A) zip(*iterables)
(B) enumerate()
(C) combine()
(D) merge()

41. What is the critical danger of using a mutable default argument in Python (e.g., `def append_to(val, target=[])`)?

(A) The default list is created once at function definition time; subsequent calls share and mutate the same list object across invocations
(B) It throws a SyntaxError
(C) It crashes the Python interpreter
(D) It converts the list to a tuple

42. What is the standard idiomatic fix for mutable default arguments in Python?

(A) Set default to None and initialize inside: `if target is None: target = []`
(B) Use `target=list()`
(C) Use `target={}`
(D) Mark parameter as static

43. What does the expression `list(map(lambda x: x*2, filter(lambda x: x%2 != 0, range(10))))` evaluate to?

(A) [2, 6, 10, 14, 18] (filters odd numbers: 1, 3, 5, 7, 9; then doubles them)
(B) [0, 4, 8, 12, 16]
(C) [1, 3, 5, 7, 9]
(D) [2, 4, 6, 8, 10]

44. What is a Generator Function in Python, and how does it differ from a standard function?

(A) A function containing the 'yield' keyword that returns a lazy generator iterator, generating values on-demand without storing the entire sequence in memory
(B) A function that generates random numbers
(C) A constructor function
(D) A function that runs on GPU

45. What happens when execution encounters a `yield` statement inside a generator function?

(A) Execution pauses, the current state and local variables are frozen in memory, and the yielded value is emitted to the caller
(B) The function terminates permanently
(C) The program reboots
(D) A new process is spawned

46. What is a Closure in Python function architecture?

(A) A nested inner function that retains access to variables in its enclosing lexical scope even after the outer function has finished executing
(B) A method that terminates a class
(C) A private module
(D) A file-closing statement

47. What is a Decorator in Python (`@decorator_name`)?

(A) A callable higher-order function that takes another function as an argument, extends or alters its behavior, and returns a modified function
(B) A tool to style terminal text
(C) A class destructor
(D) A memory profiler

48. What does the `functools.wraps` decorator preserve when authoring custom decorators?

(A) Preserves the original function's metadata (docstring, function name `__name__`, module, and signature)
(B) Preserves CPU registers
(C) Preserves global variables
(D) Prevents garbage collection

49. What is Memoization in recursive algorithms, and how does `functools.lru_cache` accelerate recursive functions?

(A) Caches the return values of expensive function calls based on input arguments, avoiding redundant re-computations of identical subproblems
(B) Compresses function bytecode
(C) Executes recursion on multiple threads
(D) Rewires recursion into while loops

50. Without memoization, what is the theoretical time complexity of computing the n-th Fibonacci number recursively `fib(n) = fib(n-1) + fib(n-2)`?

(A) O(2^n) exponential time complexity
(B) O(n) linear time
(C) O(n log n)
(D) O(1) constant time

51. With memoization or dynamic programming, what is the time complexity of computing the n-th Fibonacci number?

(A) O(n) linear time complexity with O(n) memory
(B) O(2^n)
(C) O(n^2)
(D) O(log n) with matrix exponentiation

52. What is the difference between Positional-Only and Keyword-Only parameters in Python 3.8+?

(A) Parameters before `/` must be positional-only; parameters after `*` must be passed as keyword arguments (e.g., `def f(p_only, /, standard, *, kw_only):`)
(B) Positional cannot be numbers
(C) Keyword-only must start with $
(D) There is no distinction

53. In Python function calling, what does Argument Unpacking using `*` and `**` do (e.g., `func(*args_list, **kwargs_dict)`)?

(A) `*` unpacks an iterable into individual positional arguments; `**` unpacks a dictionary into keyword arguments
(B) Multiplies arguments by 2
(C) Squares arguments
(D) Converts arguments to tuples

54. What is the result of executing the following nested function: ```python def outer(x): def inner(y): return x * y return inner f = outer(10) print(f(5)) ```?

(A) 50 (inner function forms a closure capturing x = 10; 10 * 5 = 50)
(B) 5
(C) 10
(D) TypeError

55. What happens if a function tries to modify an outer variable without `nonlocal`: ```python def outer(): count = 0 def inner(): count += 1 inner() ```?

(A) UnboundLocalError: local variable 'count' referenced before assignment
(B) count becomes 1
(C) count becomes None
(D) SyntaxError

56. How does adding `nonlocal count` inside `inner()` solve the issue in Q55?

(A) Binds 'count' to the enclosing scope variable, permitting in-place rebinding
(B) Makes count a global variable
(C) Deletes count
(D) Freezes count

57. What is the output of the loop: ```python for i in range(5): if i == 3: break else: print('Done') ```?

(A) Nothing is printed (because the loop terminated via 'break', suppressing the 'else' block)
(B) 'Done'
(C) 3
(D) 0 1 2 Done

58. What is the output if the break in Q57 is replaced with `if i == 3: continue`?

(A) 'Done' is printed (because the loop finishes all iterations without breaking)
(B) Nothing
(C) 3
(D) Error

59. What is Tail Call Optimization (TCO), and does standard CPython support it?

(A) Reusing stack frames for tail-recursive calls; standard CPython intentionally does NOT support TCO to preserve full stack trace visibility for debugging
(B) CPython optimizes all tail calls
(C) CPython converts recursion to bytecode loops
(D) CPython limits tail recursion to 10

60. How can an infinite recursive function or large recursion depth be safely evaluated in Python without stack overflow?

(A) By converting recursion into an iterative loop or using a Trampoline / custom generator stack
(B) By disabling error logging
(C) By setting recursion limit to infinity
(D) By compiling to Cython only

61. What does `zip([1, 2, 3], ['a', 'b'])` produce when converted to a list in standard Python?

(A) [(1, 'a'), (2, 'b')] (stops at the shortest input iterable)
(B) [(1, 'a'), (2, 'b'), (3, None)]
(C) Throws ValueError
(D) [(1, 'a'), (2, 'b'), (3, 'b')]

62. Which function in the `itertools` module pairs iterables of uneven length using a fill value (e.g. fillvalue=None)?

(A) itertools.zip_longest()
(B) itertools.zip_all()
(C) itertools.combine()
(D) itertools.chain()

63. What does the higher-order function `functools.partial(func, *args, **keywords)` return?

(A) A new partial object that freezes a portion of the original function's arguments, simplifying subsequent invocations
(B) The immediate return value
(C) A generator
(D) A class instance

64. How do you define a function that accepts ONLY keyword arguments in Python?

(A) Place a bare asterisk `*` as a parameter before the arguments: `def configure(*, baud_rate, parity):`
(B) Use `**kwargs` only
(C) Prefix function with `@keyword`
(D) Use `def keyword configure():`

65. What is the result of `list(range(10, 0, -2))` in Python?

(A) [10, 8, 6, 4, 2] (negative step decrements down to but excluding 0)
(B) [10, 8, 6, 4, 2, 0]
(C) [0, 2, 4, 6, 8, 10]
(D) []

66. What does the `pass` statement compile into in CPython bytecode?

(A) A NOP (No Operation) bytecode instruction that does nothing
(B) A JUMP instruction
(C) A RETURN instruction
(D) A HALT instruction

67. In digital signal threshold detection, what structure cleanly classifies signal amplitude into LOW, NORMAL, and HIGH zones?

(A) if-elif-else ladder
(B) nested while loops
(C) pass statements
(D) lambda filters

68. What does `itertools.chain('ABC', 'DEF')` generate?

(A) An iterator yielding 'A', 'B', 'C', 'D', 'E', 'F' consecutively
(B) A list of strings
(C) A 2D matrix
(D) A tuple of tuples

69. What does `itertools.cycle('AB')` do?

(A) Generates an infinite cyclic repetition: 'A', 'B', 'A', 'B', 'A', 'B'...
(B) Runs twice and stops
(C) Throws StopIteration
(D) Reverses the string

70. What does `itertools.islice(iterable, 2, 5)` do?

(A) Extracts elements from index 2 up to index 5 (exclusive) from an iterator without loading the entire stream into memory
(B) Slices a list in place
(C) Reverses indices
(D) Splits iterable into 5 chunks

71. In Python 3.10+ pattern matching, what is a 'Guard' in a case clause?

(A) An optional `if` condition appended to a pattern: `case value if value > 0:`
(B) A cryptographic checksum
(C) A memory lock
(D) A type assertion

72. What is the output of the recursive power function `def p(b, e): return 1 if e==0 else b * p(b, e-1)` for `p(3, 3)`?

(A) 27 (3 * 3 * 3 = 27)
(B) 9
(C) 81
(D) 1

73. What is the Divide-and-Conquer optimized power algorithm's time complexity for `b**n` using `p(b, n//2)`?

(A) O(log n) logarithmic time
(B) O(n) linear time
(C) O(n^2)
(D) O(1)

74. What does the built-in function `callable(obj)` check?

(A) Whether the object can be invoked as a function (returns True for functions, methods, lambdas, and classes with `__call__`)
(B) Whether object is a variable
(C) Whether object is a number
(D) Whether object is in memory

75. What is the difference between returning multiple comma-separated values `return a, b` versus returning a list `return [a, b]`?

(A) `return a, b` automatically packs the values into an immutable tuple `(a, b)`; `return [a, b]` returns a mutable list
(B) They are identical in memory
(C) Tuple return is deprecated
(D) List return is faster

76. What is the output of the classic late-binding closure loop: ```python funcs = [lambda: i for i in range(4)] print([f() for f in funcs]) ```?

(A) [3, 3, 3, 3] (closures bind to variable 'i' by reference; by the time functions are called, 'i' is 3)
(B) [0, 1, 2, 3]
(C) [0, 0, 0, 0]
(D) TypeError

77. How is the late-binding bug in Q76 solved using default arguments?

(A) funcs = [lambda i=i: i for i in range(4)] (binds current value of 'i' as a default parameter at definition time, returning [0, 1, 2, 3])
(B) Use a while loop instead
(C) Use global i
(D) Use pass statement

78. What is the space complexity of a naive non-tail recursive Fibonacci implementation `fib(n)` on the call stack?

(A) O(n) linear space complexity (maximum tree depth on the call stack is n frames)
(B) O(2^n) exponential space
(C) O(1) constant space
(D) O(n^2) quadratic space

79. How does Fast Fibonacci calculation via Matrix Exponentiation achieve O(log n) time complexity in ECE digital signal processing?

(A) By computing `[[1, 1], [1, 0]]^n` using binary exponentiation (repeated squaring) in O(log n) matrix multiplications
(B) Using FFT algorithms
(C) Using infinite Taylor series
(D) Using floating-point approximation

80. What does the `@functools.lru_cache(maxsize=128)` decorator implement internally?

(A) A Least Recently Used (LRU) dictionary-backed cache that evicts oldest unread entries when size exceeds 128
(B) A file-system database
(C) A multi-threaded queue
(D) A C++ extension module

81. What happens if you pass an unhashable mutable object (such as a list) as an argument to an `@lru_cache` decorated function?

(A) Raises a `TypeError: unhashable type: 'list'` because cache keys must be hashable tuples/primitives
(B) The list is converted to string
(C) The cache ignores the call
(D) The program crashes

82. What is a Coroutine in Python, and how does `async def` differ from a generator `def ... yield`?

(A) Coroutines (`async def`) are awaitable objects managed by an asyncio event loop; generators produce streams of values via iterators
(B) They are identical in all versions
(C) Coroutines cannot return values
(D) Generators run on threads

83. What does the expression `yield from sub_generator()` do inside a generator function?

(A) Delegates part of the generator's operations directly to another sub-generator, forwarding all values, exceptions, and returns seamlessly
(B) Terminates the sub-generator
(C) Converts sub-generator to list
(D) Loops infinitely

84. In Python function architecture, what does `func.__code__.co_varnames` contain?

(A) A tuple containing all local variable names, including function parameter names
(B) A list of global variables
(C) The bytecode hex string
(D) The function return value

85. What does `func.__code__.co_code` contain in CPython?

(A) The raw bytes representing the compiled bytecode instructions
(B) The source code string
(C) The AST tree pointer
(D) The docstring

86. Which standard library module allows disassembling Python functions into human-readable bytecode instructions?

(A) dis (import dis; dis.dis(func))
(B) inspect
(C) bytecode
(D) asm

87. What is the difference between a function closure variable `__closure__` and a global variable?

(A) Closure variables are stored in dedicated cell objects (`cell_contents`) attached to the function instance; globals reside in module dictionary
(B) Closures are stored on disk
(C) Globals cannot be changed
(D) There is no difference

88. How do you author a Parametrized Decorator in Python (a decorator that takes arguments, e.g. `@repeat(num_times=3)`)?

(A) Write a 3-tier nested function: outer function takes decorator arguments, middle function takes target function, inner wrapper executes logic
(B) Write a 2-tier function only
(C) Use class inheritance only
(D) Decorators cannot accept arguments

89. What is Function Currying, and how is it implemented in Python?

(A) Transforming a multi-argument function `f(a, b, c)` into a chain of single-argument functions `f(a)(b)(c)` using nested lambdas or closures
(B) Converting functions to C code
(C) Running functions in parallel
(D) Caching function returns

90. What is the output of `(lambda f: (lambda x: f(lambda v: x(x)(v)))(lambda x: f(lambda v: x(x)(v))))` in theoretical computer science?

(A) The Y Combinator (Fixed-Point Combinator), enabling anonymous recursion in purely functional lambda calculus
(B) A memory leak
(C) An infinite loop compiler crash
(D) The identity function

91. How does Python handle tail-call elimination when simulating state machines in telecommunication protocols?

(A) Developers use an explicit Trampoline loop yielding thunks or state functions to prevent stack overflow across thousands of state transitions
(B) CPython eliminates frames automatically
(C) By increasing RAM size
(D) By compiling to assembly

92. What is the purpose of `inspect.signature(func)` in the `inspect` module?

(A) Extracts comprehensive callable parameter specifications, default values, kind (positional/keyword), and type annotations
(B) Checks if function has bugs
(C) Measures execution time
(D) Computes memory consumption

93. What does `types.FunctionType` represent in Python's internal type system?

(A) The built-in metaclass / type of user-defined Python functions
(B) A function that has no return value
(C) A lambda function only
(D) A C-extension function

94. What is the Difference between a Method and a Function in Python?

(A) A method is a function bound to a class instance that automatically receives the instance as its first argument (`self`); a function is standalone
(B) Methods are written in C, functions in Python
(C) Functions cannot have arguments
(D) Methods cannot return values

95. What does the `@staticmethod` decorator do in Python classes?

(A) Defines a method that does not receive an implicit first argument (`self` or `cls`), behaving like a plain standalone function housed inside class namespace
(B) Makes the method run on one core
(C) Prevents method from being modified
(D) Makes method private

96. What does the `@classmethod` decorator do?

(A) Defines a method that receives the class object (`cls`) as its implicit first argument rather than an instance (`self`), ideal for alternative constructors
(B) Creates a new subclass
(C) Freezes class attributes
(D) Runs method at compile time

97. In digital filters (IIR/FIR), how is a discrete convolution sum $y[n] = \sum_{k=0}^M b_k x[n-k]$ implemented iteratively in pure Python?

(A) Using a sliding window buffer list and calculating inner products via `sum(b * x for b, x in zip(coeffs, buffer))`
(B) Using a single while loop with no buffer
(C) Using float division
(D) Using recursion only

98. What is the overhead of a Python function call compared to an inlined arithmetic operation?

(A) A Python function call allocates a new `PyFrameObject` on the call stack, incurring ~100?200 ns overhead; inlining eliminates frame allocation entirely
(B) Function calls have zero overhead
(C) Function calls take 1 second
(D) Inlined code is slower

99. What is Vectorization, and why is vectorizing loops using NumPy 100x faster than pure Python `for` loops in signal processing?

(A) Vectorized operations push looping into pre-compiled C/Fortran SIMD CPU instructions, eliminating Python interpreter type-checking and frame overhead on every element

(B) Vectorization uses quantum computing

(C) NumPy runs in the browser

(D) Python loops use disk I/O

100. What does the built-in `compile()` function do in Python?

(A) Compiles a string containing Python source code into a code object suitable for execution by `exec()` or evaluation by `eval()`

(B) Compiles Python into native C executable

(C) Generates C++ headers

(D) Minifies Python source files

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Control Flow in Python, contrasting standard conditional branching (if-elif-else) with Structural Pattern Matching (match-case) introduced in Python 3.10.

Q2. Analyze Python Iteration Structures: for loops, while loops, the range() generator, and the loop-else clause semantics.

Q3. Explain Functions in Python: definition, parameter vs argument distinction, positional, keyword, default, *args, and **kwargs.

Q4. Explain Variable Scope and the LEGB Rule in Python, detailing global, nonlocal, and closure variable bindings.

Q5. Analyze Recursion in Python: mechanics, call stack frames, base cases, recursion depth limits, and the Divide-and-Conquer paradigm.

Q6. Explain Memoization and Dynamic Programming in Python using functools.lru_cache to optimize recursive algorithms.

Q7. Explain Lambda Functions and Functional Programming primitives in Python: map(), filter(), and reduce().

Q8. Explain Python Decorators (@decorator), function wrapping, functools.wraps, and parameterized decorators.

Q9. Explain Generator Functions and the yield statement, contrasting eager memory lists with lazy generator evaluation in streaming sensor data.

Q10. Describe Iterators and the Python Iterator Protocol: __iter__() and __next__() methods, and custom iterator classes.

Q11. Explain the itertools module in Python standard library: count(), cycle(), repeat(), chain(), islice(), and combinations().

Q12. Explain the functools module: partial(), reduce(), lru_cache(), and total_ordering() with engineering implementations.

Q13. Explain Positional-Only Arguments (/) and Keyword-Only Arguments (*) in Python 3.8+ (PEP 570), discussing API design principles.

Q14. Detail Python Type Annotations and Static Type Checking (PEP 484, mypy), explaining Union, Optional, Any, and Callable types.

Q15. Analyze the Call Stack, Frame Objects (PyFrameObject), and Stack Unwinding during Exception Propagation in CPython.

Q16. Explain Closures and the Late-Binding Problem in Python loops, providing mathematical explanations and the default argument solution.

Q17. Explain the inspect and dis modules in Python standard library, demonstrating disassembly of Python bytecode opcodes.

Q18. Detail Nested Functions and Factory Functions, writing a Configurable Digital Low-Pass Filter Factory in Python.

Q19. Explain Function Overloading alternatives in Python: default arguments, variable arguments, and functools.singledispatch.

Q20. Explain the Global Interpreter Lock (GIL) in CPython and its impact on CPU-bound vs I/O-bound multithreaded Python scripts.

Q21. Write a Python script implementing a PID (Proportional-Integral-Derivative) Controller function closure with state persistence.

Q22. Write a Python recursive function to calculate the Factorial of an integer, handling edge cases, and compare with math.factorial.

Q23. Write a Python script implementing the Binary Search algorithm recursively on a sorted array of sensor frequencies.

Q24. Write a complete Python generator function to stream real-time simulated sinusoidal voltage samples `v(t) = V_peak * sin(2*pi*f*t)`.

Q25. Demonstrate how to author a custom Retry Decorator in Python with configurable attempts and backoff delay for flaky network hardware.

Q26. Write a Python program to solve the classic Tower of Hanoi problem recursively, detailing mathematical complexity.

Q27. Implement a Moving Average Filter generator function in Python that maintains a fixed sliding window buffer.

Q28. Write a Python script that uses `zip()` and dictionary comprehension to construct an Instrument Configuration table.

Q29. Explain how to write a Python Context Manager using the `@contextlib.contextmanager` decorator for timing execution blocks.

Q30. Analyze CPython bytecode disassembly of a for-loop versus a list comprehension using `dis.dis()`.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

- Both `for` and `while` loops support an optional `else` block.
- Critical Semantic Rule: The `else` block executes ONLY if the loop finishes all iterations naturally without encountering an explicit `break` statement.
- Ideal for search algorithms (e.g. searching for a threshold violation; if no item breaks the loop, `else` logs 'All samples within safe tolerance').

```python
def make_pid_controller(kp, ki, kd):
integral_error = 0.0 # Enclosed state
def compute(error, dt):
nonlocal integral_error # Modifies outer variable
integral_error += error * dt
return kp * error + ki * integral_error
return compute
```

>>Solution Question Bank Recursion and Dynamic Programming Paths Use cases Solution Code Optimize Divide and Conquer paradigm.

As N grows, the call tree doubles, making `fib(40)` require over 2 billion calls!

The Solution: Memoization via `@functools.lru_cache`:
Caches the return value of each function call mapped to its input arguments in a hash table:

```python
from functools import lru_cache

@lru_cache(maxsize=None) # Unlimited cache size
def fibonacci(n):
if n < 2: return n
return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(100)) # Computes instantly in O(N) linear time!
print(fibonacci.cache_info()) # Displays hits, misses, maxsize, currsize
```
Transforms intractable exponential algorithms into lightning-fast linear algorithms.

>> Solution to Question 3: Explain Lambda Functions and Functional Programming primitives in Python: map(), filter(), and reduce().

[Problem Statement]: Explain Python Decorators (@decorator), function wrapping, functools.wraps, and parameterized decorators.
[Detailed Step-by-Step Answer]:
A decorator is a callable that accepts a function as input, extends its behavior, and returns a modified callable wrapper:

Anatomy of a Benchmark / Execution Timer Decorator:

```python
import time
from functools import wraps

def benchmark(func):
@wraps(func) # Copies __name__, __doc__, and signature from original func
def wrapper(*args, **kwargs):
start_time = time.perf_counter()
result = func(*args, **kwargs)
end_time = time.perf_counter()
elapsed = (end_time - start_time) * 1000 # milliseconds
print(f'[BENCHMARK] {func.__name__}() took {elapsed:.4f} ms')
return result
return wrapper

@benchmark
def run_dsp_fft(samples):
"""Computes Discrete Fourier Transform."""
time.sleep(0.05) # Simulated compute
return sum(samples)
```

Parameterized Decorators (3-tier nesting):
`@repeat(num_times=3)` wraps the decorator inside an outer factory function that accepts configuration arguments and returns the actual decorator callable.

>> **Solution to Question 9:** Explain Generator Functions and the yield statement, contrasting eager memory lists with lazy generator evaluation in streaming sensor

```
# Consuming the generator with constant O(1) memory:
stream = sensor_stream_generator(100_000_000)
for sample in stream:
if sample['id'] >= 5: break
print(sample)
```

Memory footprint is strictly bounded to a single sample record, regardless of whether 10 or 10 billion samples are processed.

---

1. Infinite Iterators:
- `itertools.count(start=0, step=1)`: Infinite arithmetic progression (e.g., continuous sample numbering).
- `itertools.cycle(iterable)`: Cycles endlessly through an iterable (e.g., repeating a periodic waveform bit pattern `0b1010`).
- `itertools.repeat(elem, n)`: Repeats an element `n` times.

2. Terminating & Combinatorial Iterators:
- `itertools.chain(*iterables)`: Chains multiple distinct streams into a single contiguous stream without copying.
- `itertools.islice(iterable, start, stop, step)`: Slices an iterator lazily without consuming memory.
- `itertools.combinations(iterable, r)`: Generates all r-length combinations of elements without replacement (used for testing all resistor pairing combinations).
- `itertools.product(*iterables)`: Cartesian product (nested for-loops in a single generator).

---

1. Positional-Only (`/` marker):
- Arguments preceding `/` CANNOT be passed as keyword arguments. `func(pos_only_1=10)` raises a `TypeError`.
- Purpose: Enables C-extension compatibility, allows changing parameter names in future library versions without breaking callers, and improves performance.

2. Keyword-Only (`*` marker):
- Arguments following `*` MUST be passed by keyword name: `func(..., kw_only_1=True)`.
- Purpose: Eliminates cryptic boolean flags in call sites (e.g., `set_led(True, False, True)` vs `set_led(red=True, green=False, blink=True)`), dramatically improving code clarity in embedded APIs.

Stack Unwinding during Exceptions:

1. When an exception is raised (`raise ValueError`), the runtime checks the current frame's exception handling table (block stack).
2. If no matching `except` block is found, the current frame is popped from the call stack, its local variables are dereferenced, and control unwinds to the caller frame (`f_back`).
3. This process continues up the call stack until a matching handler is found or the top-level module is reached, whereupon CPython prints the full Traceback and exits.

Disassembly Output Analysis:
```text
2 0 RESUME 0
2 LOAD_FAST 0 (voltage)
4 LOAD_FAST 1 (current)
6 BINARY_OP 5 (*)
10 RETURN_VALUE
```
- `LOAD_FAST`: Extremely fast $O(1)$ array lookup pushing local variables onto the evaluation stack.
- `BINARY_OP 5`: Pops two operands and performs multiplication.
- `RETURN_VALUE`: Returns the top of stack to the caller.

Engineering Value: Allows verifying whether loops are optimized and understanding compiler transformations.

Unlike C++ or Java, Python does not support native function signature overloading where multiple functions share the same name with different parameter lists (subsequent `def` definitions overwrite previous ones):

Python Overloading Alternatives:
1. Default & Variable Arguments: Handling optional parameters inside a single unified function.
2. Single-Dispatch Generic Functions (`@functools.singledispatch`):
Enables function overloading based on the data type of the first argument:

```python
from functools import singledispatch

@singledispatch
def process_sensor_data(data):
raise NotImplementedError('Unsupported data type')

@process_sensor_data.register(int)
@process_sensor_data.register(float)
def _(data):
return f'Processing scalar voltage: {data:.2f} V'

@process_sensor_data.register(list)
def _(data):
return f'Processing batch array of {len(data)} samples'

print(process_sensor_data(3.3)) # Scalar voltage
print(process_sensor_data([1.1, 2.2, 3.3])) # Batch array
```

>> Solution to Question 28

[Problem Statement]: Write a Python script implementing a PID (Proportional-Integral-Derivative) Controller function closure with state persistenc
[Detailed Step-by-Step Answer]:
Here is a complete, industrial-grade discrete PID controller implementation using function closures:

```python
def make_pid_controller(kp, ki, kd, output_limits=(-100.0, 100.0)):
"""Factory returning a stateful PID controller closure."""
integral = 0.0
prev_error = 0.0
min_out, max_out = output_limits

def compute(setpoint, measured_val, dt):
nonlocal integral, prev_error
error = setpoint - measured_val

# Proportional term
p_term = kp * error

# Integral term with anti-windup clamping
integral += error * dt
i_term = ki * integral

# Derivative term (rate of change of error)
derivative = (error - prev_error) / dt if dt > 0 else 0.0
d_term = kd * derivative

# Total control output
output = p_term + i_term + d_term
# Clamp output between limits
output = max(min_out, min(max_out, output))

prev_error = error
return output

return compute

# Test PID controller
pid = make_pid_controller(kp=2.0, ki=0.5, kd=0.1)
print('Control Effort:', pid(setpoint=50.0, measured_val=40.0, dt=0.1))
```

>> Solution to Question 22: Write a Python recursive function to calculate the Factorial of an integer, handling edge cases, and compare with math.factorial.

```
  raise ValueError('Factorial is defined strictly for non-negative integers.')
  if n == 0 or n == 1: # Base cases
  return 1
  return n * recursive_factorial(n - 1) # Recursive step

  # Verify against math.factorial
  for test_val in [0, 1, 5, 10, 20]:
  res = recursive_factorial(test_val)
  expected = math.factorial(test_val)
  assert res == expected
  print(f'{test_val}! = {res}')
  ```
```

---



**>>Solution to Question 28:** a Python script implementing the Binary Search algorithm, including comments explaining each step `y = A * sin(2*pi*f*t)`.
```

```
while True:
t = n * dt
voltage = v_peak * math.sin(omega * t)
yield {'time_s': round(t, 5), 'voltage_v': round(voltage, 4)}
n += 1

# Generate first 5 samples
wave = sine_wave_generator(v_peak=10.0, freq=50.0, sample_rate=1000.0)
for _ in range(5):
sample = next(wave)
print(f't = {sample["time_s"]} s -> V = {sample["voltage_v"]} V')
```

>>Solution Question 26: Explain how to troubleshoot a network issue with confusing symptoms, like a laptop that can reach the internet but not a local file server.

The Tower of Hanoi recursive algorithm:

```python
def tower_of_hanoi(n, source, target, auxiliary):
if n == 1:
print(f'Move disk 1 from rod {source} to rod {target}')
return 1

moves = 0
# 1. Move top n-1 disks from source to auxiliary
moves += tower_of_hanoi(n - 1, source, auxiliary, target)
# 2. Move n-th disk from source to target
print(f'Move disk {n} from rod {source} to rod {target}')
moves += 1
# 3. Move n-1 disks from auxiliary to target
moves += tower_of_hanoi(n - 1, auxiliary, target, source)
return moves

total_moves = tower_of_hanoi(3, 'A', 'C', 'B')
print(f'Total Moves Required: {total_moves}') # 2^3 - 1 = 7 moves
```

Mathematical Complexity: Recurrence relation $T(n) = 2T(n-1) + 1 = 2^n - 1$. Time complexity is $O(2^n)$, and space complexity on call stack is $O(n)$.

>> Solution to Question 28: Implement a Moving Average Filter generator function in Python that maintains a fixed sliding window buffer.

[Problem Statement]: Write a Python script that uses `zip()` and dictionary comprehension to construct an Instrument Configuration table.
[Detailed Step-by-Step Answer]:
Data structure transformation using zip and comprehensions:

```python
channel_names = ['CH1_VOLT', 'CH2_CURR', 'CH3_TEMP', 'CH4_VREF']
raw_codes = [512, 128, 850, 1023]
gains = [1.0, 0.1, 0.5, 1.0]

# Construct dictionary using zip and comprehension
channel_map = {
ch: {'code': code, 'gain': gain, 'scaled_value': round(code * gain, 2)}
for ch, code, gain in zip(channel_names, raw_codes, gains)
}

import pprint
pprint.pprint(channel_map)
```

>>[Problem Statement]: Write a Python script that uses `time.perf_counter` for timing execution blocks.

```
def loop_version():
res = []
for x in range(10):
res.append(x * 2)
return res

def comp_version():
return [x * 2 for x in range(10)]

print('=== LOOP VERSION ===')
dis.dis(loop_version)
print('\n=== COMPREHENSION VERSION ===')
dis.dis(comp_version)
```

Performance Analysis:
The list comprehension uses the specialized `LIST_APPEND` bytecode opcode directly at the C level, bypassing the Python attribute lookup `res.append` on every iteration, executing roughly 30% faster than standard imperative loops.
